BANCO DE DADOS · TRABALHO PRÁTICO

Modelo Físico e Consultas SQL

Módulo de Controle de Acessos (Segurança e Permissões) de uma plataforma WEB

Aluno(a)	
Disciplina	Banco de Dados
Professor(a)	
SGBD	PostgreSQL 16
Data	Novembro de 2026

Sumário

1. Análise do domínio e regras de negócio
2. Modelo conceitual
3. Modelo físico (lógico-relacional)
4. Dicionário de dados
5. Restrições de integridade e testes
6. Consultas SQL por interface
7. Como executar
8. Apêndice A — Script DDL completo
9. Apêndice B — Dados de exemplo

1. Análise do domínio e regras de negócio

O módulo controla **quem** acessa a plataforma e **o que** cada pessoa pode fazer. O enunciado mostra três funções:

- **Login com Active Directory:** o usuário entra com a conta Microsoft. A senha fica só no AD. Para acessar, o e-mail corporativo precisa estar no grupo do AD da plataforma e cadastrado na base de dados.
- **Gerenciar usuários:** consultar, cadastrar, editar e excluir. O cadastro pede E-mail*, Grupo* (um ou mais) e Papel (opcional). A lista mostra Usuário (nome e e-mail), Grupo, Papel e Último acesso.
- **Gerenciar grupos e permissões:** consultar, cadastrar, editar e excluir grupos. Na aba Permissões, cada grupo liga ou desliga, por toggle, as permissões de cada funcionalidade (organizadas por módulo). Na aba Usuários, o grupo recebe vários usuários de uma vez.

Regras de negócio levantadas

RN1 Todo usuário pertence a **pelo menos um grupo** (o campo “Grupo *” é obrigatório).

RN2 Algumas permissões dependem de outra. Os toggles cinza da tela (ex.: “Gerenciar anexos do edital”) só podem ser ligados depois de “Visualizar editais”.

RN3 A plataforma **não armazena senha**. A autenticação é feita no AD, e a plataforma só autoriza o e-mail retornado.

RN4 O **nome** do usuário não é digitado no cadastro. A plataforma o preenche e atualiza com os dados do AD a cada autorização. Até o primeiro acesso, o nome fica vazio (“-” na lista).

RN5 O e-mail é a chave de login e é **único**, sem diferenciar maiúsculas de minúsculas.

RN6 O usuário tem acesso a todas as funcionalidades habilitadas em **qualquer** um dos seus grupos (união das permissões).

RN7 “Último acesso” é a data e hora do último login com sucesso. Se não houver, a lista mostra “Nunca acessou”.

RN8 Papel é opcional e múltiplo: um usuário pode ter zero ou mais papéis (ex.: “Analista de proposta”, “Analista técnico”).

2. Modelo conceitual

Entidades e relacionamentos identificados nas telas e nos requisitos:

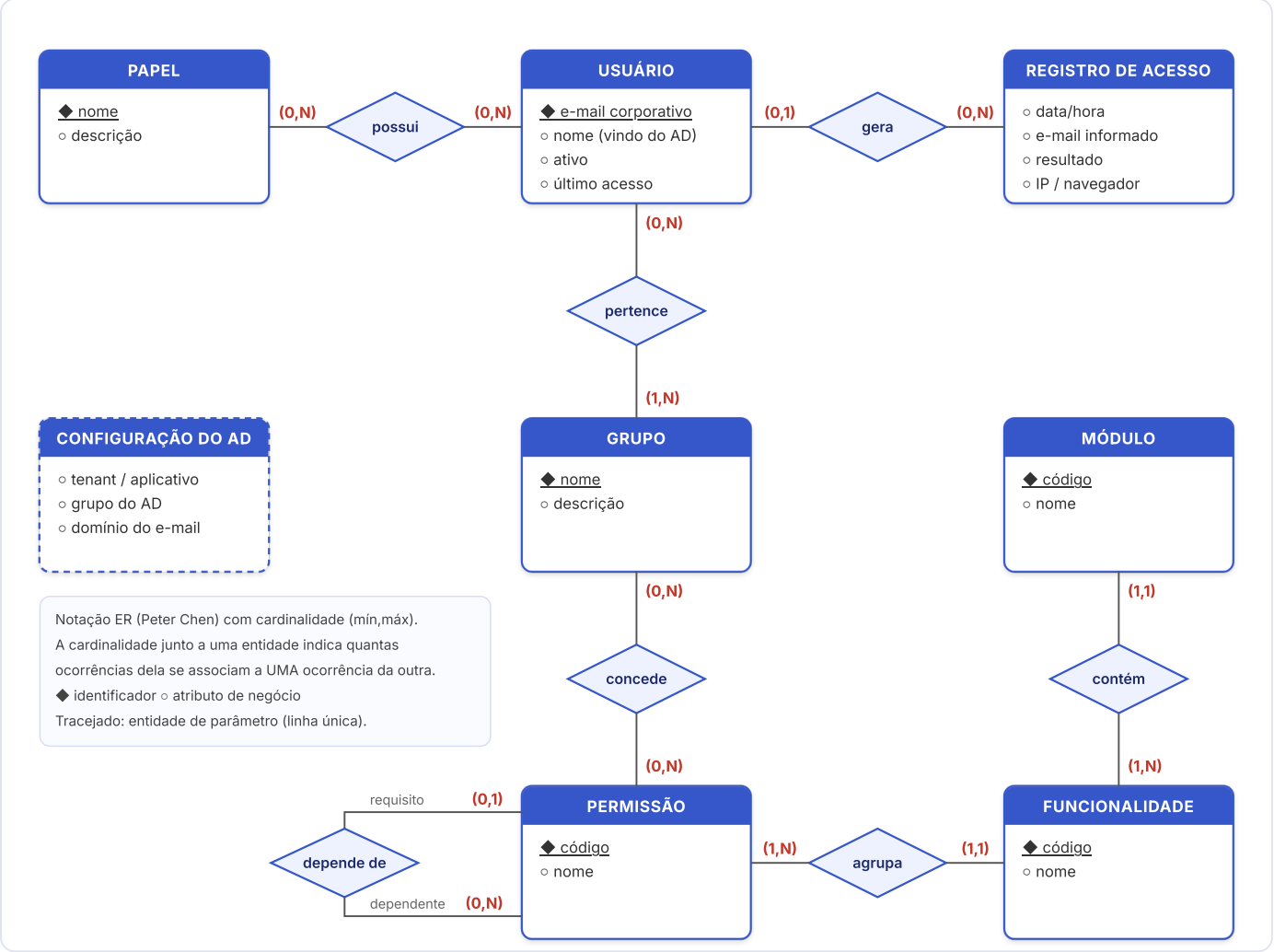


Figura 1 — Diagrama Entidade-Relacionamento (conceitual). Também está em `modelo-conceitual.png`.

- **USUÁRIO pertence a GRUPO (N:N):** no mínimo 1 grupo por usuário (RN1). Um grupo pode não ter usuários.
- **USUÁRIO possui PAPEL (N:N):** opcional nos dois lados (RN8).
- **GRUPO concede PERMISSÃO (N:N):** cada ocorrência corresponde a um toggle ligado.
- **MÓDULO contém FUNCIONALIDADE** e **FUNCIONALIDADE agrupa PERMISSÃO (1:N):** é a hierarquia exibida na tela “Adicionar grupo” (EDITAIS › Seleção de editais › Visualizar editais).
- **PERMISSÃO depende de PERMISSÃO (auto-relacionamento 1:N):** implementa a RN2.
- **USUÁRIO gera REGISTRO DE ACESSO (1:N):** é a auditoria do login. A tentativa de um e-mail não cadastrado também é registrada, sem usuário associado.
- **CONFIGURAÇÃO DO AD:** é uma entidade de parâmetro, com uma única linha (tenant, aplicativo e grupo do AD que libera o acesso).

3. Modelo físico (lógico-relacional)

Implementação em **PostgreSQL 16**, no schema `controle_acesso`. Cada relacionamento N:N virou uma tabela associativa com chave primária composta.

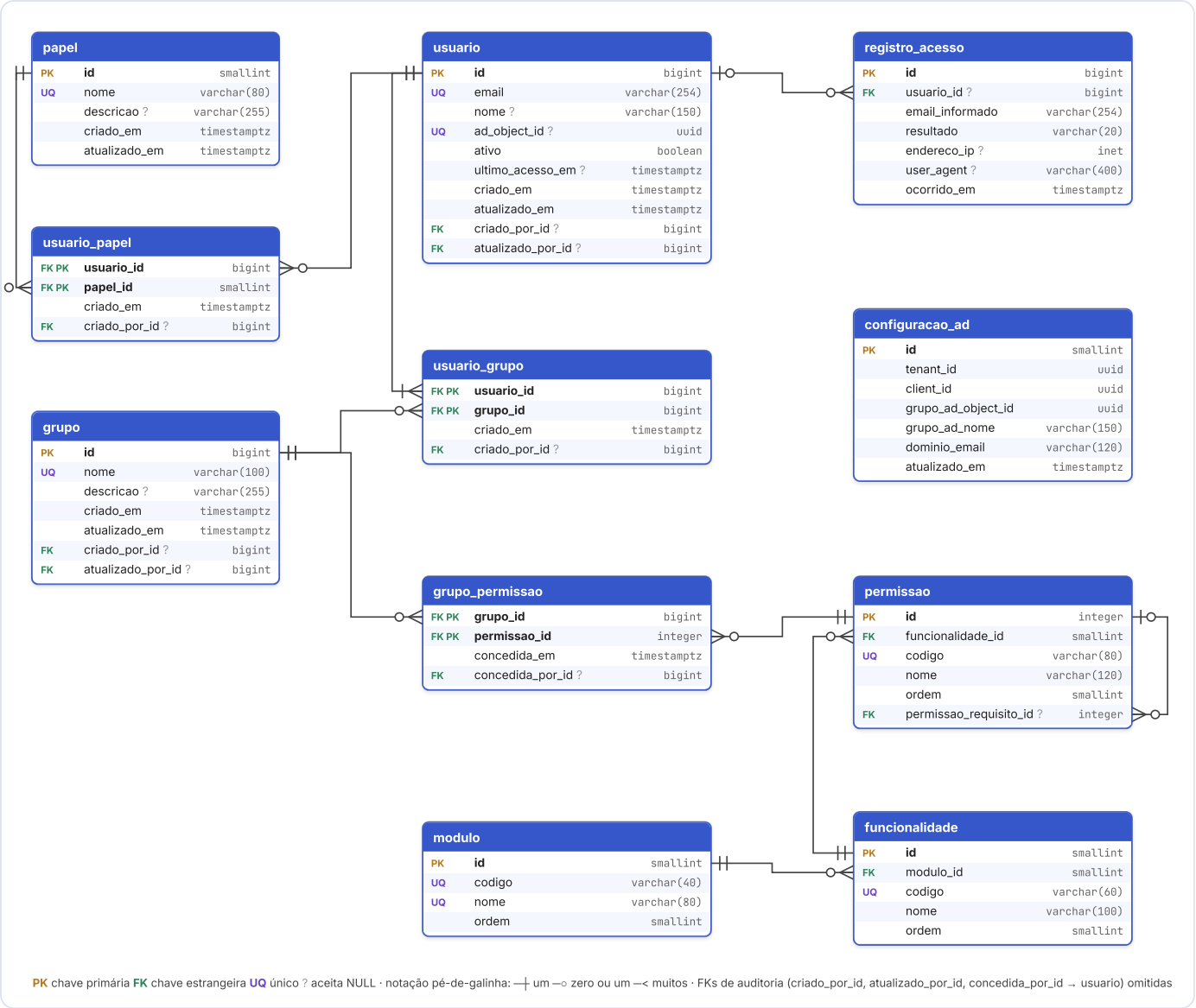


Figura 2 — Modelo físico: tabelas, colunas, tipos e chaves. Também está em `modelo-fisico.png`.

Decisões de projeto

- **Chaves substitutas** `GENERATED ALWAYS AS IDENTITY` em todas as entidades. As chaves naturais (e-mail, nome do grupo, códigos) são garantidas por `UNIQUE`.
- **E-mail único sem diferenciar maiúsculas:** índice único em `lower(email)`, mais um `CHECK` de formato.
- **Tipos:** `timestamptz` para datas (com fuso horário), `uuid` para os identificadores do AD, `inet` para IP, `boolean` para flags e `varchar(n)` com limites realistas (254 é o máximo de um e-mail).
- **Atributos técnicos de auditoria:** `criado_em`, `atualizado_em` (mantido por trigger), `criado_por_id` e `atualizado_por_id`.
- **Exclusão:** excluir um usuário ou grupo remove os vínculos em cascata (`ON DELETE CASCADE`). O catálogo (módulo, funcionalidade, permissão, papel em uso) é protegido com `RESTRICT`. A auditoria é preservada com `SET NULL`.

- **RN1 e RN2** não cabem em PK, FK ou CHECK. Por isso viraram `CONSTRAINT TRIGGER … DEFERRABLE INITIALLY DEFERRED`, verificados no `COMMIT`. Assim é possível inserir o usuário e os grupos dele, ou o grupo e as permissões dele, na mesma transação.
- **Sem coluna de senha** (RN3). O *client secret* do AD fica no cofre de segredos da aplicação, não no banco.

4. Dicionário de dados

Gerado do catálogo do PostgreSQL. As descrições estão gravadas no próprio banco com `COMMENT ON`. A coluna **Natureza** separa atributos de Negócio (vêm do domínio) de atributos Técnicos (chaves substitutas, auditoria, integração e ordenação).

configuracao_ad

Parâmetros do Active Directory usados no login (linha única)

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
id	smallint	Não	PK	CHECK; DEFAULT 1	Técnico	Identificador fixo da linha única (sempre 1)
tenant_id	uuid	Não			Técnico	ID do diretório (tenant) da empresa no Microsoft Entra ID / AD
client_id	uuid	Não			Técnico	ID do aplicativo da plataforma registrado no AD
grupo_ad_object_id	uuid	Não			Técnico	ID do grupo do AD cujos membros podem acessar a plataforma
grupo_ad_nome	varchar(150)	Não			Negócio	Nome do grupo do AD (exibição)
dominio_email	varchar(120)	Não		CHECK	Negócio	Domínio do e-mail corporativo
atualizado_em	timestamptz	Não		DEFAULT now()	Técnico	Data/hora da última alteração (mantida por trigger)

ck_configuracao_ad_dominio	CHECK	CHECK (((dominio_email)::text ~ '^[a-z0-9.-]+\.[a-z]{2,}$'::text))
ck_configuracao_ad_linha_unica	CHECK	CHECK ((id = 1))
configuracao_ad_pkey	PRIMARY KEY	PRIMARY KEY (id)

usuario

Usuário autorizado a acessar a plataforma (autenticado pelo AD; sem senha local)

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
id	bigint	Não	PK	IDENTITY	Técnico	Chave primária (identificador gerado automaticamente)
email	varchar(254)	Não	UQ	CHECK; UNIQUE (lower)	Negócio	E-mail corporativo; chave de login (único, sem diferenciar maiúsculas)
nome	varchar(150)	Sim		CHECK	Negócio	Nome vindo do AD; preenchido/atualizado pela plataforma a cada login
ad_object_id	uuid	Sim	UQ	UNIQUE	Técnico	Identificador do usuário no AD (claim "oid" do token)
ativo	boolean	Não		DEFAULT true	Negócio	Indica se o acesso está liberado (bloqueio sem excluir)

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
ultimo_acesso_em	timestamptz	Sim			Negócio	Data/hora do último login com sucesso; NULL = nunca acessou
criado_em	timestamptz	Não		DEFAULT now()	Técnico	Data/hora de criação do registro
atualizado_em	timestamptz	Não		DEFAULT now()	Técnico	Data/hora da última alteração (mantida por trigger)
criado_por_id	bigint	Sim	FK	FK → usuario	Técnico	Usuário que criou o registro
atualizado_por_id	bigint	Sim	FK	FK → usuario	Técnico	Usuário que fez a última alteração

ck_usuario_email_formato	CHECK	CHECK (((email)::text ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'::text))
ck_usuario_nome_vazio	CHECK	CHECK (((nome IS NULL) OR (btrim((nome)::text) <> ''::text)))
fk_usuario_atualizado_por	FOREIGN KEY	FOREIGN KEY (atualizado_por_id) REFERENCES controle_acesso.usuario(id) ON DELETE SET NULL
fk_usuario_criado_por	FOREIGN KEY	FOREIGN KEY (criado_por_id) REFERENCES controle_acesso.usuario(id) ON DELETE SET NULL
usuario_pkey	PRIMARY KEY	PRIMARY KEY (id)
uq_usuario_ad_object_id	UNIQUE	UNIQUE (ad_object_id)
ix_usuario_nome_lower	INDEX	(lower((nome)::text))
uq_usuario_email_lower	UNIQUE INDEX	(lower((email)::text))

papel

Papel/cargo que o usuário exerce (ex.: Analista de proposta); opcional

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
id	smallint	Não	PK	IDENTITY	Técnico	Chave primária (identificador gerado automaticamente)
nome	varchar(80)	Não	UQ	UNIQUE (lower)	Negócio	Nome do papel exibido na coluna PAPEL
descricao	varchar(255)	Sim			Negócio	Descrição do papel
criado_em	timestamptz	Não		DEFAULT now()	Técnico	Data/hora de criação do registro
atualizado_em	timestamptz	Não		DEFAULT now()	Técnico	Data/hora da última alteração (mantida por trigger)

papel_pkey	PRIMARY KEY	PRIMARY KEY (id)
uq_papel_nome_lower	UNIQUE INDEX	(lower((nome)::text))

usuario_papel

Papéis atribuídos a cada usuário (N:N)

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
usuario_id	bigint	Não	FK PK	FK → usuario	Negócio	Usuário do vínculo
papel_id	smallint	Não	FK PK	FK → papel	Negócio	Papel atribuído ao usuário

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
criado_em	timestamptz	Não		DEFAULT now()	Técnico	Data/hora de criação do registro
criado_por_id	bigint	Sim	FK	FK → usuario	Técnico	Usuário que criou o registro

fk_usuario_papel_criado_por	FOREIGN KEY	FOREIGN KEY (criado_por_id) REFERENCES controle_acesso.usuario(id) ON DELETE SET NULL
fk_usuario_papel_papel	FOREIGN KEY	FOREIGN KEY (papel_id) REFERENCES controle_acesso.papel(id) ON DELETE RESTRICT
fk_usuario_papel_usuario	FOREIGN KEY	FOREIGN KEY (usuario_id) REFERENCES controle_acesso.usuario(id) ON DELETE CASCADE
pk_usuario_papel	PRIMARY KEY	PRIMARY KEY (usuario_id, papel_id)
ix_usuario_papel_papel	INDEX	(papel_id)

grupo

Grupo de usuários que concentra permissões de acesso

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
id	bigint	Não	PK	IDENTITY	Técnico	Chave primária (identificador gerado automaticamente)
nome	varchar(100)	Não	UQ	CHECK; UNIQUE (lower)	Negócio	Nome do grupo ("Nome do grupo" na tela)
descricao	varchar(255)	Sim			Negócio	Descrição do grupo
criado_em	timestamptz	Não		DEFAULT now()	Técnico	Data/hora de criação do registro
atualizado_em	timestamptz	Não		DEFAULT now()	Técnico	Data/hora da última alteração (mantida por trigger)
criado_por_id	bigint	Sim	FK	FK → usuario	Técnico	Usuário que criou o registro
atualizado_por_id	bigint	Sim	FK	FK → usuario	Técnico	Usuário que fez a última alteração

ck_grupo_nome_vazio	CHECK	CHECK ((btrim((nome)::text) <> ''::text))
fk_grupo_atualizado_por	FOREIGN KEY	FOREIGN KEY (atualizado_por_id) REFERENCES controle_acesso.usuario(id) ON DELETE SET NULL
fk_grupo_criado_por	FOREIGN KEY	FOREIGN KEY (criado_por_id) REFERENCES controle_acesso.usuario(id) ON DELETE SET NULL
grupo_pkey	PRIMARY KEY	PRIMARY KEY (id)
uq_grupo_nome_lower	UNIQUE INDEX	(lower((nome)::text))

usuario_grupo

Usuários vinculados a cada grupo (N:N); todo usuário tem pelo menos 1 grupo

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
usuario_id	bigint	Não	FK PK	FK → usuario	Negócio	Usuário do vínculo
grupo_id	bigint	Não	FK PK	FK → grupo	Negócio	Grupo ao qual o usuário pertence
criado_em	timestamptz	Não		DEFAULT now()	Técnico	Data/hora de criação do registro
criado_por_id	bigint	Sim	FK	FK → usuario	Técnico	Usuário que criou o registro

fk_usuario_grupo_criado_por	FOREIGN KEY	FOREIGN KEY (criado_por_id) REFERENCES controle_acesso.usuario(id) ON DELETE SET NULL
fk_usuario_grupo_grupo	FOREIGN KEY	FOREIGN KEY (grupo_id) REFERENCES controle_acesso.grupo(id) ON DELETE CASCADE
fk_usuario_grupo_usuario	FOREIGN KEY	FOREIGN KEY (usuario_id) REFERENCES controle_acesso.usuario(id) ON DELETE CASCADE
pk_usuario_grupo	PRIMARY KEY	PRIMARY KEY (usuario_id, grupo_id)
ix_usuario_grupo_grupo	INDEX	(grupo_id)

modulo

Módulo do sistema exibido como seção (ex.: EDITAIS, CONTRATOS)

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
id	smallint	Não	PK	IDENTITY	Técnico	Chave primária (identificador gerado automaticamente)
codigo	varchar(40)	Não	UQ	CHECK; UNIQUE	Técnico	Código estável do módulo, usado pela aplicação
nome	varchar(80)	Não	UQ	UNIQUE	Negócio	Nome do módulo exibido como seção na tela de grupo
ordem	smallint	Não		DEFAULT 0	Técnico	Ordem de exibição na tela

ck_modulo_codigo	CHECK	CHECK (((codigo)::text ~ '^[A-Z0-9_]+$'::text))
modulo_pkey	PRIMARY KEY	PRIMARY KEY (id)
uq_modulo_codigo	UNIQUE	UNIQUE (codigo)
uq_modulo_nome	UNIQUE	UNIQUE (nome)

funcionalidade

Funcionalidade de um módulo (ex.: Seleção de editais)

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
id	smallint	Não	PK	IDENTITY	Técnico	Chave primária (identificador gerado automaticamente)
modulo_id	smallint	Não	FK	FK → modulo; UNIQUE (modulo_id, nome)	Negócio	Módulo ao qual a funcionalidade pertence
codigo	varchar(60)	Não	UQ	CHECK; UNIQUE	Técnico	Código estável da funcionalidade, usado pela aplicação
nome	varchar(100)	Não		UNIQUE (modulo_id, nome)	Negócio	Nome exibido como título do bloco de toggles
ordem	smallint	Não		DEFAULT 0	Técnico	Ordem de exibição na tela

ck_funcionalidade_codigo	CHECK	CHECK (((codigo)::text ~ '^[A-Z0-9_]+$'::text))
fk_funcionalidade_modulo	FOREIGN KEY	FOREIGN KEY (modulo_id) REFERENCES controle_acesso.modulo(id) ON DELETE RESTRICT
funcionalidade_pkey	PRIMARY KEY	PRIMARY KEY (id)
uq_funcionalidade_codigo	UNIQUE	UNIQUE (codigo)
uq_funcionalidade_modulo_nome	UNIQUE	UNIQUE (modulo_id, nome)

permissao

Ação habilitável por toggle na tela de grupo (ex.: Visualizar editais)

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
id	integer	Não	PK	CHECK; IDENTITY	Técnico	Chave primária (identificador gerado automaticamente)
funcionalidade_id	smallint	Não	FK	FK → funcionalidade; UNIQUE (funcionalidade_id, nome)	Negócio	Funcionalidade à qual a permissão pertence
codigo	varchar(80)	Não	UQ	CHECK; UNIQUE	Técnico	Código verificado pelo back-end (ex.: EDITAL_VISUALIZAR)
nome	varchar(120)	Não		UNIQUE (funcionalidade_id, nome)	Negócio	Texto exibido ao lado do toggle
ordem	smallint	Não		DEFAULT 0	Técnico	Ordem de exibição na tela
permissao_requisito_id	integer	Sim	FK	CHECK; FK → permissao	Negócio	Permissão que precisa estar habilitada antes desta

ck_permissao_codigo	CHECK	CHECK (((codigo)::text ~ '^[A-Z0-9_]+$'::text))
ck_permissao_requisito	CHECK	CHECK ((permissao_requisito_id <> id))
fk_permissao_funcionalidade	FOREIGN KEY	FOREIGN KEY (funcionalidade_id) REFERENCES controle_acesso.funcionalidade(id) ON DELETE RESTRICT
fk_permissao_requisito	FOREIGN KEY	FOREIGN KEY (permissao_requisito_id) REFERENCES controle_acesso.permissao(id) ON DELETE RESTRICT
permissao_pkey	PRIMARY KEY	PRIMARY KEY (id)
uq_permissao_codigo	UNIQUE	UNIQUE (codigo)
uq_permissao_funcionalidade_nome	UNIQUE	UNIQUE (funcionalidade_id, nome)
ix_permissao_requisito	INDEX	(permissao_requisito_id)

grupo_permissao

Permissões habilitadas em cada grupo (linha presente = toggle ligado)

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
grupo_id	bigint	Não	FK PK	FK → grupo	Negócio	Grupo que recebe a permissão
permissao_id	integer	Não	FK PK	FK → permissao	Negócio	Permissão habilitada no grupo
concedida_em	timestamptz	Não		DEFAULT now()	Técnico	Data/hora em que a permissão foi habilitada
concedida_por_id	bigint	Sim	FK	FK → usuario	Técnico	Usuário que habilitou a permissão

fk_grupo_permissao_concedida_por	FOREIGN KEY	FOREIGN KEY (concedida_por_id) REFERENCES controle_acesso.usuario(id) ON DELETE SET NULL

fk_grupo_permissao_grupo	FOREIGN KEY	FOREIGN KEY (grupo_id) REFERENCES controle_acesso.grupo(id) ON DELETE CASCADE
fk_grupo_permissao_permissao	FOREIGN KEY	FOREIGN KEY (permissao_id) REFERENCES controle_acesso.permissao(id) ON DELETE CASCADE
pk_grupo_permissao	PRIMARY KEY	PRIMARY KEY (grupo_id, permissao_id)
ix_grupo_permissao_permissao	INDEX	(permissao_id)

registro_acesso

Auditoria das tentativas de login

Coluna	Tipo	Nulo	Chave	Restrições	Natureza	Descrição
id	bigint	Não	PK	IDENTITY	Técnico	Chave primária (identificador gerado automaticamente)
usuario_id	bigint	Sim	FK	FK → usuario	Negócio	Usuário identificado; NULL se o e-mail não estiver cadastrado
email_informado	varchar(254)	Não			Negócio	E-mail retornado pelo AD na tentativa
resultado	varchar(20)	Não		CHECK	Negócio	SUCESSO, NAO_CADASTRADO, FORA_GRUPO_AD ou INATIVO
endereco_ip	inet	Sim			Técnico	IP de origem da tentativa
user_agent	varchar(400)	Sim			Técnico	Navegador/dispositivo de origem
ocorrido_em	timestamptz	Não		DEFAULT now()	Negócio	Data/hora da tentativa

ck_registro_acesso_resultado	CHECK	CHECK (((resultado)::text = ANY ((ARRAY['SUCESSO'::character varying, 'NAO_CADASTRADO'::character varying, 'FORA_GRUPO_AD'::character varying, 'INATIVO'::character varying])::text[])))
fk_registro_acesso_usuario	FOREIGN KEY	FOREIGN KEY (usuario_id) REFERENCES controle_acesso.usuario(id) ON DELETE SET NULL
registro_acesso_pkey	PRIMARY KEY	PRIMARY KEY (id)
ix_registro_acesso_usuario_data	INDEX	(usuario_id, ocorrido_em DESC)

5. Restrições de integridade e testes

Resumo das restrições usadas:

Tipo	Onde	Garante
PRIMARY KEY	Todas as tabelas (composta nas associativas)	Identidade da linha; impede vínculo duplicado
FOREIGN KEY	Associativas, catálogo e auditoria	Integridade referencial (CASCADE, RESTRICT ou SET NULL)
UNIQUE	e-mail (lower), nome do grupo e do papel (lower), códigos, ad_object_id	Unicidade das chaves naturais
CHECK	formato do e-mail, códigos em MAIÚSCULAS, resultado do login, configuração com linha única, nomes não vazios	Domínio dos valores
NOT NULL / DEFAULT	Atributos obrigatórios	Preenchimento obrigatório e valores iniciais
CONSTRAINT TRIGGER (RN1)	usuario, usuario_grupo	Todo usuário tem ao menos 1 grupo
CONSTRAINT TRIGGER (RN2)	grupo_permissao	A permissão só fica habilitada se a permissão-requisito também estiver

Testes executados (`04_testes_integridade.sql`)

Cada comando abaixo tenta violar uma regra. A última coluna é a mensagem real que o PostgreSQL devolveu ao rejeitar o comando.

#	Restrição	Tentativa	Resposta do SGBD
T01	**PK**	Vincular o mesmo usuário duas vezes ao mesmo grupo	`duplicate key value violates unique constraint "pk_usuario_grupo"`
T02	**UNIQUE**	E-mail repetido, mesmo com letras maiúsculas	`duplicate key value violates unique constraint "uq_usuario_email_lower"`
T03	**CHECK**	E-mail em formato inválido	`new row for relation "usuario" violates check constraint "ck_usuario_email_formato"`
T04	**FK**	Vincular um grupo inexistente	`insert or update on table "usuario_grupo" violates foreign key constraint "fk_usuario_grupo_grupo"`
T05	**FK RESTRICT**	Excluir um papel que ainda está em uso	`update or delete on table "papel" violates foreign key constraint "fk_usuario_papel_papel" on table "usuario_papel"`
T06	**RN1**	Cadastrar usuário sem nenhum grupo ("Grupo *" é obrigatório)	`O usuário 13 deve pertencer a pelo menos um grupo`
T07	**RN1**	Remover o único grupo de um usuário	`O usuário 14 deve pertencer a pelo menos um grupo`
T08	**RN2**	Habilitar permissão sem habilitar a permissão-requisito	`A permissão "Editar contrato" exige que sua permissão-requisito esteja habilitada no grupo`

#	Restrição	Tentativa	Resposta do SGBD
T09	**RN2**	Desabilitar "Visualizar editais" mantendo as permissões dependentes	`A permissão "Adicionar edital manualmente" exige que sua permissão-requisito esteja habilitada no grupo`
T10	**CHECK**	Segunda linha na configuração do AD (tabela de linha única)	`new row for relation "configuracao_ad" violates check constraint "ck_configuracao_ad_linha_unica"`

6. Consultas SQL por interface

Cada necessidade de consulta das telas virou uma instrução SQL. Os parâmetros aparecem como variáveis do psql (`:'email'`, `:grupo_id`...). Na aplicação, eles viram parâmetros preparados (`$1`, `$2`), o que evita SQL injection. Os resultados mostrados são reais: o arquivo `03_consultas.sql` foi executado em sequência sobre os dados de exemplo.

Valores de exemplo dos parâmetros: e-mail `adailton@positivo.com.br`, usuário logado e administrador = 1 (Adailton), grupo consultado = 4 (Seleção de editais), busca de usuários = `positivo`, busca de grupos = `editais`, novo usuário `maria.lima@positivo.com.br` e novo grupo "Análise de propostas".

Interface	Necessidade de consulta	Consulta
Login com Active Directory	Parâmetros do Active Directory	C01
	Autorizar o e-mail retornado pelo AD	C02
	Registrar login com sucesso e atualizar o nome vindo do AD	C03
	Registrar tentativa de login negada	C04
	Permissões efetivas do usuário (carregadas na sessão)	C05
	Menu lateral: módulos e funcionalidades liberados	C06
	O usuário tem a permissão X?	C07
Permissões › Usuários	Listagem com busca e paginação	C08
	Opções do campo "Grupo *"	C09
	Opções do campo "Papel"	C10
	Validar se o e-mail já está cadastrado	C11
	Inserir usuário com grupos e papéis	C12
	Carregar o usuário no formulário	C13
	Salvar e-mail, grupos e papéis	C14
	Excluir usuário (ícone da lixeira)	C15
Permissões › Grupos	Listagem com busca	C16
	Usuários do grupo (ícone de usuários)	C17
Adicionar / Editar grupo	Toggles agrupados por módulo e funcionalidade	C18
	Usuários com checkbox e grupos atuais	C19
	Validar se o nome do grupo já existe	C20
	Inserir grupo com permissões e usuários	C21
	Salvar nome, permissões (toggles) e usuários	C22
	Usuários que ficariam sem nenhum grupo	C23
	Excluir o grupo	C24

6.1 Login com Active Directory

Botão “Entrar com Microsoft”: autenticação no AD, autorização na plataforma, sessão e menu.

C01 Parâmetros do Active Directory

Monta o redirecionamento para a Microsoft e informa qual grupo do AD deve constar no token ("groups") para o acesso ser permitido.

```
SELECT tenant_id, client_id, grupo_ad_object_id, grupo_ad_nome, dominio_email
  FROM configuracao_ad
 WHERE id = 1;
```

RESULTADO COM OS DADOS DE EXEMPLO

tenant_id	client_id	grupo_ad_object_id	grupo_ad_nome	dominio_email
6f1d2c3b-0000-4000-8000-000000000001	6f1d2c3b-0000-4000-8000-000000000002	6f1d2c3b-0000-4000-8000-000000000003	GRP-PLATAFORMA-EDITAIS	positivo.com.br

1 linha(s)

C02 Autorizar o e-mail retornado pelo AD

Depois que o AD autentica (a senha fica só no AD), a plataforma confere se o e-mail corporativo está cadastrado e ativo. Nenhuma linha retornada = acesso negado.

```
SELECT u.id, u.email, u.nome, u.ativo
  FROM usuario u
 WHERE lower(u.email) = lower(:'email');
```

RESULTADO COM OS DADOS DE EXEMPLO

id	email	nome	ativo
1	adailton@positivo.com.br	Adailton Araújo	t

1 linha(s)

C03 Registrar login com sucesso e atualizar o nome vindo do AD

O nome não é digitado no cadastro: a plataforma o atualiza a cada autorização.

```
WITH atualizado AS (
  UPDATE usuario
     SET nome             = :'nome_ad',
         ad_object_id     = :'oid_ad'::uuid,
         ultimo_acesso_em = now()
   WHERE lower(email) = lower(:'email')
     AND ativo
  RETURNING id, email, nome, ultimo_acesso_em
), registro AS (
  INSERT INTO registro_acesso (usuario_id, email_informado, resultado, endereco_ip, user_agent)
  SELECT id, email, 'SUCESSO', :'ip'::inet, :'user_agent' FROM atualizado
  RETURNING id
)
SELECT a.id AS usuario_id, a.nome, a.email, a.ultimo_acesso_em
  FROM atualizado a;
```

RESULTADO COM OS DADOS DE EXEMPLO

usuario_id	nome	email	ultimo_acesso_em
1	Adailton Araújo	adailton@positivo.com.br	2026-11-03 22:58:24.230783+00

1 linha(s)

C04 Registrar tentativa de login negada

resultado: NAO_CADASTRADO (e-mail fora da base), FORA_GRUPO_AD ou INATIVO.

```
INSERT INTO registro_acesso (usuario_id, email_informado, resultado, endereco_ip, user_agent)
SELECT (SELECT id FROM usuario WHERE lower(email) = lower('visitante@gmail.com')),
       'visitante@gmail.com', 'NAO_CADASTRADO', :'ip'::inet, :'user_agent'
RETURNING id, email_informado, resultado, ocorrido_em;
```

RESULTADO COM OS DADOS DE EXEMPLO

id	email_informado	resultado	ocorrido_em
7	visitante@gmail.com	NAO_CADASTRADO	2026-11-03 22:58:24.233045+00

1 linha(s)

INSERT 0 1

C05 Permissões efetivas do usuário (carregadas na sessão)

O usuário acessa tudo que estiver habilitado em QUALQUER um dos seus grupos.

```
SELECT DISTINCT p.codigo, p.nome
  FROM usuario_grupo   ug
  JOIN grupo_permissao gp ON gp.grupo_id = ug.grupo_id
  JOIN permissao       p  ON p.id = gp.permissao_id
 WHERE ug.usuario_id = :usuario_id
 ORDER BY p.codigo;
```

RESULTADO COM OS DADOS DE EXEMPLO

codigo	nome
ANALISE_GERENCIAR	Gerenciar análise de edital
ANALISE_GERENCIAR_ETAPAS	Gerenciar etapas do edital
ANALISE_GERENCIAR_HISTORICO	Gerenciar histórico de análise de edital
ANALISE_PRE_TECNICA	Realizar pré-análise técnica
ANALISE_REALIZAR	Realizar análise de edital
ANALISE_REALIZAR_PROPOSTA	Realizar proposta
ANALISE_VISUALIZAR_HISTORICO	Visualizar histórico de análise de edital
CNPJ_GERENCIAR_CONSULTA	Gerenciar consulta
CONTRATO_CADASTRAR	Cadastrar contrato
CONTRATO_EDITAR	Editar contrato
CONTRATO_VISUALIZAR	Visualizar contratos
EDITAL_ADICIONAR_MANUAL	Adicionar edital manualmente
EDITAL_GERENCIAR_ANEXOS	Gerenciar anexos do edital
EDITAL_GERENCIAR_GRUPOS	Gerenciar grupos de editais
EDITAL_GERENCIAR_STATUS	Gerenciar status
EDITAL_VISUALIZAR	Visualizar editais
RECEBER_REGISTRAR	Registrar recebimento
RECEBER_VISUALIZAR	Visualizar contas a receber

18 linha(s)

C06 Menu lateral: módulos e funcionalidades liberados

```
SELECT m.nome AS modulo, f.nome AS funcionalidade, count(DISTINCT p.id) AS permissoes
  FROM usuario_grupo   ug
  JOIN grupo_permissao gp ON gp.grupo_id = ug.grupo_id
  JOIN permissao       p  ON p.id = gp.permissao_id
```

```
  JOIN funcionalidade  f  ON f.id = p.funcionalidade_id
  JOIN modulo          m  ON m.id = f.modulo_id
 WHERE ug.usuario_id = :usuario_id
 GROUP BY m.ordem, m.nome, f.ordem, f.nome
 ORDER BY m.ordem, f.ordem;
```

RESULTADO COM OS DADOS DE EXEMPLO

modulo	funcionalidade	permissoes
Editais	Seleção de editais	5
Editais	Análise de editais	7
Contratos	Consulta de CNPJ	1
Contratos	Cadastro de contratos	3
Financeiro	Contas a receber	2

5 linha(s)

C07 O usuário tem a permissão X?

Usada pelo back-end antes de cada ação protegida.

```
SELECT EXISTS (
  SELECT 1
    FROM usuario         u
    JOIN usuario_grupo   ug ON ug.usuario_id = u.id
    JOIN grupo_permissao gp ON gp.grupo_id = ug.grupo_id
    JOIN permissao       p  ON p.id = gp.permissao_id
   WHERE u.id = :usuario_id
     AND u.ativo
     AND p.codigo = :'permissao'
) AS autorizado;
```

RESULTADO COM OS DADOS DE EXEMPLO

autorizado
t

1 linha(s)

6.2 Permissões › Usuários

Listagem com busca, modal “Adicionar usuário” (E-mail, Grupo*, Papel), edição e exclusão.

C08 Listagem com busca e paginação

Colunas: USUÁRIO (nome + e-mail), GRUPO, PAPEL, ÚLTIMO ACESSO. "Buscar por usuário" filtra por nome ou e-mail; total_registros serve à paginação.

```
SELECT u.id,
       COALESCE(u.nome, '-')            AS nome,
       u.email,
       COALESCE(g.grupos, '')           AS grupos,
       COALESCE(p.papeis, '-')          AS papeis,
       fn_tempo_desde(u.ultimo_acesso_em) AS ultimo_acesso,
       count(*) OVER ()                 AS total_registros
  FROM usuario u
  LEFT JOIN LATERAL (
       SELECT string_agg(gr.nome, ', ' ORDER BY gr.nome) AS grupos
         FROM usuario_grupo ug JOIN grupo gr ON gr.id = ug.grupo_id
        WHERE ug.usuario_id = u.id) g ON true
  LEFT JOIN LATERAL (
       SELECT string_agg(pa.nome, ', ' ORDER BY pa.nome) AS papeis
         FROM usuario_papel up JOIN papel pa ON pa.id = up.papel_id
        WHERE up.usuario_id = u.id) p ON true
 WHERE :'busca' = ''
    OR u.nome  ILIKE '%' || :'busca' || '%'
    OR u.email ILIKE '%' || :'busca' || '%'
 ORDER BY u.nome NULLS LAST, u.email
 LIMIT :limite OFFSET :deslocamento;
```

RESULTADO COM OS DADOS DE EXEMPLO

id	nome	email	grupos	papeis	ultimo_acesso	total_registros
1	Adailton Araújo	adailton@positivo.com.br	Cadastro de contratos, Contas a receber, Liderança de editais, Seleção de editais, Validação de CNPJ	Analista de documentos, Analista de proposta, Analista técnico	Agora	5
4	Djulia dos Santos	djuliads@positivo.com.br	Cadastro de contratos, Seleção de editais	Analista técnico	1 dia	5
2	Laís Cândido	laishc@positivo.com.br	Contas a receber, Seleção de editais, Validação de CNPJ	Analista de proposta	6 horas	5
3	Rodrigo Mendes	usuario32024@positivo.com	Cadastro de contratos, Validação de CNPJ	Analista de documentos	30 minutos	5
5	-	guilhermesousa@positivo.com.br	Cadastro de contratos, Seleção de editais	-	Nunca acessou	5

5 linha(s)

C09 Opções do campo "Grupo *"

```
SELECT id, nome FROM grupo ORDER BY nome;
```

RESULTADO COM OS DADOS DE EXEMPLO

id	nome
1	Cadastro de contratos
2	Contas a receber

3	Liderança de editais
4	Seleção de editais
5	Validação de CNPJ

5 linha(s)

C10 Opções do campo "Papel"

```
SELECT id, nome FROM papel ORDER BY nome;
```

RESULTADO COM OS DADOS DE EXEMPLO

id	nome
3	Analista de documentos
1	Analista de proposta
2	Analista técnico

3 linha(s)

C11 Validar se o e-mail já está cadastrado

```
SELECT EXISTS (SELECT 1 FROM usuario WHERE lower(email) = lower(btrim(:'novo_email'))) AS
email_ja_cadastrado;
```

RESULTADO COM OS DADOS DE EXEMPLO

email_ja_cadastrado
f

1 linha(s)

C12 Inserir usuário com grupos e papéis

Um único comando: o trigger adiado RN1 confere no COMMIT se há pelo menos 1 grupo.

```
WITH novo AS (
  INSERT INTO usuario (email, criado_por_id, atualizado_por_id)
  VALUES (lower(btrim(:'novo_email')), :admin_id, :admin_id)
  RETURNING id, email
), grupos AS (
  INSERT INTO usuario_grupo (usuario_id, grupo_id, criado_por_id)
  SELECT novo.id, g, :admin_id FROM novo, unnest(:'grupos'::bigint[]) AS g
  RETURNING grupo_id
), papeis AS (
  INSERT INTO usuario_papel (usuario_id, papel_id, criado_por_id)
  SELECT novo.id, p, :admin_id FROM novo, unnest(:'papeis'::smallint[]) AS p
  RETURNING papel_id
)
SELECT novo.id, novo.email,
       (SELECT count(*) FROM grupos) AS grupos_vinculados,
       (SELECT count(*) FROM papeis) AS papeis_vinculados
  FROM novo
\gset novo_
SELECT :novo_id AS id_criado;
```

RESULTADO COM OS DADOS DE EXEMPLO

id_criado
6

1 linha(s)

C13 Carregar o usuário no formulário

```
SELECT u.id, u.email, u.nome,
       ARRAY(SELECT grupo_id FROM usuario_grupo WHERE usuario_id = u.id ORDER BY grupo_id) AS
grupo_ids,
       ARRAY(SELECT papel_id FROM usuario_papel WHERE usuario_id = u.id ORDER BY papel_id) AS
papel_ids
  FROM usuario u
 WHERE u.id = :usuario_id;
```

RESULTADO COM OS DADOS DE EXEMPLO

id	email	nome	grupo_ids	papel_ids
1	adailton@positivo.com.br	Adailton Araújo	{1,2,3,4,5}	{1,2,3}

1 linha(s)

C14 Salvar e-mail, grupos e papéis

Sincroniza os vínculos: remove os desmarcados e insere os novos. (no exemplo, edita o usuário criado em C12)

```
BEGIN;
UPDATE usuario
   SET email = lower(btrim(:'email_edicao')), atualizado_por_id = :admin_id
 WHERE id = :novo_id;

DELETE FROM usuario_grupo
 WHERE usuario_id = :novo_id AND grupo_id <> ALL (:'grupos_edicao'::bigint[]);
INSERT INTO usuario_grupo (usuario_id, grupo_id, criado_por_id)
SELECT :novo_id, g, :admin_id FROM unnest(:'grupos_edicao'::bigint[]) AS g
ON CONFLICT DO NOTHING;

DELETE FROM usuario_papel
 WHERE usuario_id = :novo_id AND papel_id <> ALL (:'papeis_edicao'::smallint[]);
INSERT INTO usuario_papel (usuario_id, papel_id, criado_por_id)
SELECT :novo_id, p, :admin_id FROM unnest(:'papeis_edicao'::smallint[]) AS p
ON CONFLICT DO NOTHING;
COMMIT;
```

RESULTADO COM OS DADOS DE EXEMPLO

BEGIN

UPDATE 1

DELETE 1

INSERT 0 1

DELETE 1

INSERT 0 2

COMMIT

C15 Excluir usuário (ícone da lixeira)

Os vínculos com grupos e papéis são removidos em cascata (ON DELETE CASCADE); o histórico em registro_acesso é mantido (ON DELETE SET NULL). (no exemplo, exclui o usuário criado em C12)

```
DELETE FROM usuario
 WHERE id = :novo_id
RETURNING id, email;
```

RESULTADO COM OS DADOS DE EXEMPLO

id	email
6	maria.silva@positivo.com.br

1 linha(s)

DELETE 1

6.3 Permissões › Grupos

Listagem de grupos com quantidade de permissões e de usuários.

C16 Listagem com busca

Colunas: GRUPO, PERMISSÕES (habilitadas), USUÁRIOS (vinculados).

```
SELECT g.id, g.nome,
       (SELECT count(*) FROM grupo_permissao gp WHERE gp.grupo_id = g.id) AS permissoes,
       (SELECT count(*) FROM usuario_grupo  ug WHERE ug.grupo_id = g.id) AS usuarios
  FROM grupo g
 WHERE :'busca_grupo' = '' OR g.nome ILIKE '%' || :'busca_grupo' || '%'
 ORDER BY g.nome;
```

RESULTADO COM OS DADOS DE EXEMPLO

id	nome	permissoes	usuarios
3	Liderança de editais	12	1
4	Seleção de editais	5	4

2 linha(s)

C17 Usuários do grupo (ícone de usuários)

```
SELECT u.id, COALESCE(u.nome, '-') AS nome, u.email
  FROM usuario_grupo ug
  JOIN usuario u ON u.id = ug.usuario_id
 WHERE ug.grupo_id = :grupo_id
 ORDER BY u.nome NULLS LAST, u.email;
```

RESULTADO COM OS DADOS DE EXEMPLO

id	nome	email
1	Adailton Araújo	adailton@positivo.com.br
4	Djulia dos Santos	djuliads@positivo.com.br
2	Laís Cândido	laishc@positivo.com.br
5	-	guilhermesousa@positivo.com.br

4 linha(s)

6.4 Adicionar / Editar grupo

Aba Permissões (toggles por módulo e funcionalidade) e aba Usuários (vincular vários usuários).

C18 Toggles agrupados por módulo e funcionalidade

habilitada = estado do toggle; bloqueada = o toggle fica cinza porque a permissão-requisito não está habilitada. Para "Adicionar grupo", use grupo_id = 0 (nenhum toggle habilitado).

```
SELECT m.nome  AS modulo,
       f.nome  AS funcionalidade,
       p.id    AS permissao_id,
       p.nome  AS permissao,
       (gp.permissao_id IS NOT NULL) AS habilitada,
       (p.permissao_requisito_id IS NOT NULL AND req.permissao_id IS NULL) AS bloqueada
  FROM permissao p
  JOIN funcionalidade f ON f.id = p.funcionalidade_id
  JOIN modulo         m ON m.id = f.modulo_id
  LEFT JOIN grupo_permissao gp  ON gp.permissao_id  = p.id
                               AND gp.grupo_id      = :grupo_id
  LEFT JOIN grupo_permissao req ON req.permissao_id = p.permissao_requisito_id
                               AND req.grupo_id     = :grupo_id
 ORDER BY m.ordem, f.ordem, p.ordem;
```

RESULTADO COM OS DADOS DE EXEMPLO

modulo	funcionalidade	permissao_id	permissao	habilitada	bloqueada
Editais	Seleção de editais	1	Visualizar editais	t	f
Editais	Seleção de editais	2	Gerenciar anexos do edital	t	f
Editais	Seleção de editais	3	Gerenciar status	t	f
Editais	Seleção de editais	4	Adicionar edital manualmente	t	f
Editais	Seleção de editais	5	Gerenciar grupos de editais	t	f
Editais	Análise de editais	6	Gerenciar análise de edital	f	f
Editais	Análise de editais	7	Visualizar histórico de análise de edital	f	f
Editais	Análise de editais	8	Realizar pré-análise técnica	f	f
Editais	Análise de editais	9	Realizar análise de edital	f	f
Editais	Análise de editais	10	Gerenciar etapas do edital	f	f
Editais	Análise de editais	11	Realizar proposta	f	f
Editais	Análise de editais	12	Gerenciar histórico de análise de edital	f	t
Contratos	Consulta de CNPJ	13	Gerenciar consulta	f	f
Contratos	Cadastro de contratos	14	Visualizar contratos	f	f
Contratos	Cadastro de contratos	15	Cadastrar contrato	f	t
Contratos	Cadastro de contratos	16	Editar contrato	f	t
Contratos	Cadastro de contratos	17	Excluir contrato	f	t
Financeiro	Contas a receber	18	Visualizar contas a receber	f	f
Financeiro	Contas a receber	19	Registrar recebimento	f	t

19 linha(s)

C19 Usuários com checkbox e grupos atuais

marcado = o usuário já pertence ao grupo editado.

```
SELECT u.id,
       COALESCE(u.nome, split_part(u.email, '@', 1)) AS nome,
       u.email,
```

```
       EXISTS (SELECT 1 FROM usuario_grupo x
               WHERE x.usuario_id = u.id AND x.grupo_id = :grupo_id) AS marcado,
       (SELECT string_agg(g.nome, ', ' ORDER BY g.nome)
          FROM usuario_grupo ug JOIN grupo g ON g.id = ug.grupo_id
         WHERE ug.usuario_id = u.id) AS grupos
  FROM usuario u
 WHERE u.ativo
 ORDER BY nome;
```

RESULTADO COM OS DADOS DE EXEMPLO

id	nome	email	marcado	grupos
1	Adailton Araújo	adailton@positivo.com.br	t	Cadastro de contratos, Contas a receber, Liderança de editais, Seleção de editais, Validação de CNPJ
4	Djulia dos Santos	djuliads@positivo.com.br	t	Cadastro de contratos, Seleção de editais
2	Laís Cândido	laishc@positivo.com.br	t	Contas a receber, Seleção de editais, Validação de CNPJ
3	Rodrigo Mendes	usuario32024@positivo.com	f	Cadastro de contratos, Validação de CNPJ
5	guilhermesousa	guilhermesousa@positivo.com.br	t	Cadastro de contratos, Seleção de editais

5 linha(s)

C20 Validar se o nome do grupo já existe

```
SELECT EXISTS (SELECT 1 FROM grupo WHERE lower(nome) = lower(btrim(:'nome_grupo'))) AS
nome_ja_existe;
```

RESULTADO COM OS DADOS DE EXEMPLO

nome_ja_existe
f

1 linha(s)

C21 Inserir grupo com permissões e usuários

O trigger adiado RN2 garante que os requisitos das permissões também foram habilitados.

```
WITH novo AS (
  INSERT INTO grupo (nome, criado_por_id, atualizado_por_id)
  VALUES (btrim(:'nome_grupo'), :admin_id, :admin_id)
  RETURNING id, nome
), perms AS (
  INSERT INTO grupo_permissao (grupo_id, permissao_id, concedida_por_id)
  SELECT novo.id, p, :admin_id FROM novo, unnest(:'permissoes'::int[]) AS p
  RETURNING permissao_id
), membros AS (
  INSERT INTO usuario_grupo (usuario_id, grupo_id, criado_por_id)
  SELECT u, novo.id, :admin_id FROM novo, unnest(:'usuarios'::bigint[]) AS u
  RETURNING usuario_id
)
SELECT novo.id, novo.nome,
       (SELECT count(*) FROM perms)   AS permissoes,
       (SELECT count(*) FROM membros) AS usuarios
  FROM novo
\gset novo_grupo_
SELECT :novo_grupo_id AS id_criado;
```

RESULTADO COM OS DADOS DE EXEMPLO

id_criado
6

1 linha(s)

C22 Salvar nome, permissões (toggles) e usuários

(no exemplo, edita o grupo criado em C21)

```
BEGIN;
UPDATE grupo
   SET nome = btrim(:'nome_grupo_edicao'), atualizado_por_id = :admin_id
 WHERE id = :novo_grupo_id;

DELETE FROM grupo_permissao
 WHERE grupo_id = :novo_grupo_id AND permissao_id <> ALL (:'permissoes_edicao'::int[]);
INSERT INTO grupo_permissao (grupo_id, permissao_id, concedida_por_id)
SELECT :novo_grupo_id, p, :admin_id FROM unnest(:'permissoes_edicao'::int[]) AS p
ON CONFLICT DO NOTHING;

DELETE FROM usuario_grupo
 WHERE grupo_id = :novo_grupo_id AND usuario_id <> ALL (:'usuarios_edicao'::bigint[]);
INSERT INTO usuario_grupo (usuario_id, grupo_id, criado_por_id)
SELECT u, :novo_grupo_id, :admin_id FROM unnest(:'usuarios_edicao'::bigint[]) AS u
ON CONFLICT DO NOTHING;
COMMIT;
```

RESULTADO COM OS DADOS DE EXEMPLO

BEGIN

UPDATE 1

DELETE 2

INSERT 0 1

DELETE 1

INSERT 0 0

COMMIT

C23 Usuários que ficariam sem nenhum grupo

Exibido na confirmação; se retornar linhas, a exclusão é barrada pela RN1. (no exemplo, verifica o grupo criado em C21)

```
SELECT u.id, COALESCE(u.nome, '-') AS nome, u.email
  FROM usuario_grupo ug
  JOIN usuario u ON u.id = ug.usuario_id
 WHERE ug.grupo_id = :novo_grupo_id
   AND NOT EXISTS (SELECT 1 FROM usuario_grupo o
                   WHERE o.usuario_id = ug.usuario_id AND o.grupo_id <> ug.grupo_id);
```

RESULTADO COM OS DADOS DE EXEMPLO

id	nome	email

0 linha(s)

C24 Excluir o grupo

Permissões e vínculos com usuários são removidos em cascata. (no exemplo, exclui o grupo criado em C21)

```
DELETE FROM grupo
 WHERE id = :novo_grupo_id
```

```
RETURNING id, nome;
```

RESULTADO COM OS DADOS DE EXEMPLO

id	nome
6	Análise de propostas e editais

1 linha(s)

DELETE 1

7. Como executar

```
createdb controle_acesso
psql -d controle_acesso -f sql/01_modelo_fisico.sql       -- cria schema, tabelas, restrições e
triggers
psql -d controle_acesso -f sql/02_dados_exemplo.sql       -- carrega os dados das telas
psql -d controle_acesso -f sql/03_consultas.sql           -- executa as consultas C01-C24
psql -d controle_acesso -f sql/04_testes_integridade.sql  -- cada teste deve FALHAR com o erro da
restrição
```

Apêndice A — Script DDL completo (`01_modelo_fisico.sql`)

```sql
-- ============================================================================
--  Módulo de Controle de Acessos de uma plataforma WEB
--  Modelo físico — PostgreSQL 16
--
--  Convenções
--    * schema  : controle_acesso
--    * nomes   : snake_case, singular; PK "id"; FK "<tabela>_id"
--    * técnicos: criado_em / atualizado_em / criado_por_id / atualizado_por_id
--    * a SENHA NÃO É ARMAZENADA: a autenticação é feita no Active Directory
--      (Microsoft Entra ID) via OpenID Connect. A plataforma guarda apenas o
--      e-mail corporativo autorizado e os dados vindos do token do AD.
-- ============================================================================

DROP SCHEMA IF EXISTS controle_acesso CASCADE;
CREATE SCHEMA controle_acesso;
SET search_path TO controle_acesso;

-- ----------------------------------------------------------------------------
-- Função utilitária: mantém "atualizado_em" sempre correto
-- ----------------------------------------------------------------------------
CREATE FUNCTION fn_atualiza_timestamp() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  NEW.atualizado_em := now();
  RETURN NEW;
END $$;

-- ============================================================================
-- 1. CONFIGURAÇÃO DO ACTIVE DIRECTORY (tabela de linha única)
--    Identifica o tenant e o GRUPO do AD cujos membros podem acessar.
--    O "client secret" fica no cofre de segredos da aplicação, nunca aqui.
-- ============================================================================
CREATE TABLE configuracao_ad (
  id                 smallint     PRIMARY KEY DEFAULT 1,
  tenant_id          uuid         NOT NULL,
  client_id          uuid         NOT NULL,
  grupo_ad_object_id uuid         NOT NULL,
  grupo_ad_nome      varchar(150) NOT NULL,
  dominio_email      varchar(120) NOT NULL,            -- ex.: positivo.com.br
  atualizado_em      timestamptz  NOT NULL DEFAULT now(),
  CONSTRAINT ck_configuracao_ad_linha_unica CHECK (id = 1),
  CONSTRAINT ck_configuracao_ad_dominio     CHECK (dominio_email ~ '^[a-z0-9.-]+\.[a-z]{2,}$')
);
CREATE TRIGGER tg_configuracao_ad_atualizado BEFORE UPDATE ON configuracao_ad
  FOR EACH ROW EXECUTE FUNCTION fn_atualiza_timestamp();

-- ============================================================================
-- 2. USUÁRIO
--    "nome" não é digitado no cadastro: é preenchido/atualizado a partir
--    do AD a cada login (por isso é NULL até o primeiro acesso).
-- ============================================================================
CREATE TABLE usuario (
  id                bigint       GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email             varchar(254) NOT NULL,
  nome              varchar(150),
  ad_object_id      uuid,                              -- "oid" do usuário no AD
  ativo             boolean      NOT NULL DEFAULT true,
  ultimo_acesso_em  timestamptz,                       -- NULL = "Nunca acessou"
  criado_em         timestamptz  NOT NULL DEFAULT now(),
  atualizado_em     timestamptz  NOT NULL DEFAULT now(),
  criado_por_id     bigint,
  atualizado_por_id bigint,
  CONSTRAINT ck_usuario_email_formato CHECK (email ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'),
```

```
  CONSTRAINT ck_usuario_nome_vazio    CHECK (nome IS NULL OR btrim(nome) <> ''),
  CONSTRAINT uq_usuario_ad_object_id  UNIQUE (ad_object_id),
  CONSTRAINT fk_usuario_criado_por     FOREIGN KEY (criado_por_id)
             REFERENCES usuario (id) ON DELETE SET NULL,
  CONSTRAINT fk_usuario_atualizado_por FOREIGN KEY (atualizado_por_id)
             REFERENCES usuario (id) ON DELETE SET NULL
);
-- e-mail único sem diferenciar maiúsculas/minúsculas (é a chave de login)
CREATE UNIQUE INDEX uq_usuario_email_lower ON usuario (lower(email));
-- busca da tela "Buscar por usuário" (nome ou e-mail)
CREATE INDEX ix_usuario_nome_lower ON usuario (lower(nome));
CREATE TRIGGER tg_usuario_atualizado BEFORE UPDATE ON usuario
  FOR EACH ROW EXECUTE FUNCTION fn_atualiza_timestamp();

-- ======================================================================
-- 3. PAPEL (ex.: Analista de proposta, Analista técnico)
--    Opcional para o usuário (0..N).
-- ======================================================================
CREATE TABLE papel (
  id             smallint     GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome           varchar(80)  NOT NULL,
  descricao      varchar(255),
  criado_em      timestamptz  NOT NULL DEFAULT now(),
  atualizado_em  timestamptz  NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX uq_papel_nome_lower ON papel (lower(nome));
CREATE TRIGGER tg_papel_atualizado BEFORE UPDATE ON papel
  FOR EACH ROW EXECUTE FUNCTION fn_atualiza_timestamp();

-- Associativa USUÁRIO x PAPEL (N:N)
CREATE TABLE usuario_papel (
  usuario_id     bigint      NOT NULL,
  papel_id       smallint    NOT NULL,
  criado_em      timestamptz NOT NULL DEFAULT now(),
  criado_por_id  bigint,
  CONSTRAINT pk_usuario_papel PRIMARY KEY (usuario_id, papel_id),
  CONSTRAINT fk_usuario_papel_usuario FOREIGN KEY (usuario_id)
             REFERENCES usuario (id) ON DELETE CASCADE,
  CONSTRAINT fk_usuario_papel_papel   FOREIGN KEY (papel_id)
             REFERENCES papel (id)   ON DELETE RESTRICT,
  CONSTRAINT fk_usuario_papel_criado_por FOREIGN KEY (criado_por_id)
             REFERENCES usuario (id) ON DELETE SET NULL
);
CREATE INDEX ix_usuario_papel_papel ON usuario_papel (papel_id);

-- ======================================================================
-- 4. GRUPO DE USUÁRIOS
-- ======================================================================
CREATE TABLE grupo (
  id                bigint       GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome              varchar(100) NOT NULL,
  descricao         varchar(255),
  criado_em         timestamptz  NOT NULL DEFAULT now(),
  atualizado_em     timestamptz  NOT NULL DEFAULT now(),
  criado_por_id     bigint,
  atualizado_por_id  bigint,
  CONSTRAINT ck_grupo_nome_vazio CHECK (btrim(nome) <> ''),
  CONSTRAINT fk_grupo_criado_por     FOREIGN KEY (criado_por_id)
             REFERENCES usuario (id) ON DELETE SET NULL,
  CONSTRAINT fk_grupo_atualizado_por FOREIGN KEY (atualizado_por_id)
             REFERENCES usuario (id) ON DELETE SET NULL
);
CREATE UNIQUE INDEX uq_grupo_nome_lower ON grupo (lower(nome));
CREATE TRIGGER tg_grupo_atualizado BEFORE UPDATE ON grupo
  FOR EACH ROW EXECUTE FUNCTION fn_atualiza_timestamp();

-- Associativa USUÁRIO x GRUPO (N:N). Todo usuário deve ter ≥ 1 grupo.
CREATE TABLE usuario_grupo (
```

```
  usuario_id     bigint      NOT NULL,
  grupo_id       bigint      NOT NULL,
  criado_em      timestamptz NOT NULL DEFAULT now(),
  criado_por_id  bigint,
  CONSTRAINT pk_usuario_grupo PRIMARY KEY (usuario_id, grupo_id),
  CONSTRAINT fk_usuario_grupo_usuario FOREIGN KEY (usuario_id)
             REFERENCES usuario (id) ON DELETE CASCADE,
  CONSTRAINT fk_usuario_grupo_grupo   FOREIGN KEY (grupo_id)
             REFERENCES grupo (id)   ON DELETE CASCADE,
  CONSTRAINT fk_usuario_grupo_criado_por FOREIGN KEY (criado_por_id)
             REFERENCES usuario (id) ON DELETE SET NULL
);
CREATE INDEX ix_usuario_grupo_grupo ON usuario_grupo (grupo_id);

-- ========================================================================
-- 5. CATÁLOGO DE FUNCIONALIDADES DO SISTEMA
--    MÓDULO (EDITAIS, CONTRATOS…) 1:N FUNCIONALIDADE (Seleção de editais…)
--    FUNCIONALIDADE 1:N PERMISSÃO (Visualizar editais, Gerenciar status…)
-- ========================================================================
CREATE TABLE modulo (
  id      smallint    GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  codigo  varchar(40) NOT NULL,
  nome    varchar(80) NOT NULL,
  ordem   smallint    NOT NULL DEFAULT 0,
  CONSTRAINT uq_modulo_codigo UNIQUE (codigo),
  CONSTRAINT uq_modulo_nome   UNIQUE (nome),
  CONSTRAINT ck_modulo_codigo CHECK (codigo ~ '^[A-Z0-9_]+$')
);

CREATE TABLE funcionalidade (
  id         smallint    GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  modulo_id  smallint    NOT NULL,
  codigo     varchar(60) NOT NULL,
  nome       varchar(100) NOT NULL,
  ordem      smallint    NOT NULL DEFAULT 0,
  CONSTRAINT uq_funcionalidade_codigo      UNIQUE (codigo),
  CONSTRAINT uq_funcionalidade_modulo_nome UNIQUE (modulo_id, nome),
  CONSTRAINT ck_funcionalidade_codigo CHECK (codigo ~ '^[A-Z0-9_]+$'),
  CONSTRAINT fk_funcionalidade_modulo FOREIGN KEY (modulo_id)
             REFERENCES modulo (id) ON DELETE RESTRICT
);

-- "permissao_requisito_id": permissão que precisa estar habilitada antes
-- (na tela, os toggles ficam desabilitados até habilitar "Visualizar editais").
CREATE TABLE permissao (
  id                    integer      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  funcionalidade_id     smallint     NOT NULL,
  codigo                varchar(80)  NOT NULL,          -- usado no código da aplicação
  nome                  varchar(120) NOT NULL,          -- texto exibido no toggle
  ordem                 smallint     NOT NULL DEFAULT 0,
  permissao_requisito_id  integer,
  CONSTRAINT uq_permissao_codigo             UNIQUE (codigo),
  CONSTRAINT uq_permissao_funcionalidade_nome UNIQUE (funcionalidade_id, nome),
  CONSTRAINT ck_permissao_codigo    CHECK (codigo ~ '^[A-Z0-9_]+$'),
  CONSTRAINT ck_permissao_requisito CHECK (permissao_requisito_id <> id),
  CONSTRAINT fk_permissao_funcionalidade FOREIGN KEY (funcionalidade_id)
             REFERENCES funcionalidade (id) ON DELETE RESTRICT,
  CONSTRAINT fk_permissao_requisito FOREIGN KEY (permissao_requisito_id)
             REFERENCES permissao (id) ON DELETE RESTRICT
);
CREATE INDEX ix_permissao_requisito ON permissao (permissao_requisito_id);

-- Associativa GRUPO x PERMISSÃO (N:N). Linha presente = toggle HABILITADO.
CREATE TABLE grupo_permissao (
  grupo_id         bigint      NOT NULL,
  permissao_id     integer     NOT NULL,
  concedida_em     timestamptz NOT NULL DEFAULT now(),
  concedida_por_id  bigint,
```

```
  CONSTRAINT pk_grupo_permissao PRIMARY KEY (grupo_id, permissao_id),
  CONSTRAINT fk_grupo_permissao_grupo     FOREIGN KEY (grupo_id)
             REFERENCES grupo (id)     ON DELETE CASCADE,
  CONSTRAINT fk_grupo_permissao_permissao FOREIGN KEY (permissao_id)
             REFERENCES permissao (id) ON DELETE CASCADE,
  CONSTRAINT fk_grupo_permissao_concedida_por FOREIGN KEY (concedida_por_id)
             REFERENCES usuario (id)   ON DELETE SET NULL
);
CREATE INDEX ix_grupo_permissao_permissao ON grupo_permissao (permissao_id);

-- ======================================================================
-- 6. REGISTRO DE ACESSO (auditoria das tentativas de login)
-- ======================================================================
CREATE TABLE registro_acesso (
  id               bigint       GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  usuario_id       bigint,                                  -- NULL se e-mail não cadastrado
  email_informado  varchar(254) NOT NULL,
  resultado        varchar(20)  NOT NULL,
  endereco_ip      inet,
  user_agent       varchar(400),
  ocorrido_em      timestamptz  NOT NULL DEFAULT now(),
  CONSTRAINT ck_registro_acesso_resultado CHECK (resultado IN
             ('SUCESSO', 'NAO_CADASTRADO', 'FORA_GRUPO_AD', 'INATIVO')),
  CONSTRAINT fk_registro_acesso_usuario FOREIGN KEY (usuario_id)
             REFERENCES usuario (id) ON DELETE SET NULL
);
CREATE INDEX ix_registro_acesso_usuario_data ON registro_acesso (usuario_id, ocorrido_em DESC);

-- ======================================================================
-- 7. REGRAS DE INTEGRIDADE QUE NÃO CABEM EM PK/FK/CHECK
--    Implementadas como CONSTRAINT TRIGGER DEFERRABLE: a verificação ocorre
--    no COMMIT, permitindo inserir o usuário e seus grupos na mesma transação.
-- ======================================================================

-- RN1: todo usuário deve pertencer a pelo menos um grupo ("Grupo *" obrigatório)
CREATE FUNCTION fn_valida_usuario_tem_grupo() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
  v_usuario_id bigint;
BEGIN
  IF TG_TABLE_NAME = 'usuario' THEN
    v_usuario_id := NEW.id;
  ELSE
    v_usuario_id := OLD.usuario_id;
  END IF;

  IF EXISTS (SELECT 1 FROM usuario WHERE id = v_usuario_id)
     AND NOT EXISTS (SELECT 1 FROM usuario_grupo WHERE usuario_id = v_usuario_id) THEN
    RAISE EXCEPTION 'O usuário % deve pertencer a pelo menos um grupo', v_usuario_id
      USING ERRCODE = 'check_violation';
  END IF;
  RETURN NULL;
END $$;

CREATE CONSTRAINT TRIGGER ct_usuario_tem_grupo
  AFTER INSERT ON usuario DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION fn_valida_usuario_tem_grupo();

CREATE CONSTRAINT TRIGGER ct_usuario_grupo_remocao
  AFTER DELETE OR UPDATE ON usuario_grupo DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION fn_valida_usuario_tem_grupo();

-- RN2: uma permissão só pode estar habilitada no grupo se a sua
--      permissão-requisito também estiver habilitada no mesmo grupo.
CREATE FUNCTION fn_valida_requisito_permissao() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
  v_grupo_id bigint;
```

```
  v_nome     text;
BEGIN
  IF TG_OP = 'DELETE' THEN
    v_grupo_id := OLD.grupo_id;
  ELSE
    v_grupo_id := NEW.grupo_id;
  END IF;

  SELECT p.nome INTO v_nome
    FROM grupo_permissao gp
    JOIN permissao p ON p.id = gp.permissao_id
   WHERE gp.grupo_id = v_grupo_id
     AND p.permissao_requisito_id IS NOT NULL
     AND NOT EXISTS (SELECT 1 FROM grupo_permissao r
                     WHERE r.grupo_id = v_grupo_id
                       AND r.permissao_id = p.permissao_requisito_id)
   LIMIT 1;
  IF v_nome IS NOT NULL THEN
    RAISE EXCEPTION 'A permissão "%" exige que sua permissão-requisito esteja habilitada no grupo', v_nome
      USING ERRCODE = 'check_violation';
  END IF;
  RETURN NULL;
END $$;

CREATE CONSTRAINT TRIGGER ct_grupo_permissao_requisito
  AFTER INSERT OR UPDATE OR DELETE ON grupo_permissao DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION fn_valida_requisito_permissao();

-- ========================================================================
-- 8. FUNÇÃO DE APOIO ÀS CONSULTAS: texto da coluna "ÚLTIMO ACESSO"
--    (ex.: "30 minutos", "3 horas", "1 dia", "Nunca acessou")
-- ========================================================================
CREATE FUNCTION fn_tempo_desde(p_momento timestamptz) RETURNS text
LANGUAGE sql STABLE AS $$
  SELECT CASE
    WHEN p_momento IS NULL THEN 'Nunca acessou'
    WHEN now() - p_momento < interval '1 minute' THEN 'Agora'
    WHEN now() - p_momento < interval '1 hour' THEN
         n_min || CASE WHEN n_min = 1 THEN ' minuto' ELSE ' minutos' END
    WHEN now() - p_momento < interval '1 day' THEN
         n_h   || CASE WHEN n_h = 1 THEN ' hora' ELSE ' horas' END
    ELSE n_d   || CASE WHEN n_d = 1 THEN ' dia' ELSE ' dias' END
  END
  FROM (SELECT floor(extract(epoch FROM now() - p_momento) / 60)::int    AS n_min,
               floor(extract(epoch FROM now() - p_momento) / 3600)::int  AS n_h,
               floor(extract(epoch FROM now() - p_momento) / 86400)::int AS n_d) t
$$;

-- ========================================================================
-- 9. DICIONÁRIO DE DADOS (comentários gravados no catálogo do banco)
-- ========================================================================
COMMENT ON TABLE configuracao_ad IS 'Parâmetros do Active Directory usados no login (linha única)';
COMMENT ON TABLE usuario         IS 'Usuário autorizado a acessar a plataforma (autenticado pelo AD; sem
senha local)';
COMMENT ON TABLE papel           IS 'Papel/cargo que o usuário exerce (ex.: Analista de proposta); opcional';
COMMENT ON TABLE usuario_papel   IS 'Papéis atribuídos a cada usuário (N:N)';
COMMENT ON TABLE grupo           IS 'Grupo de usuários que concentra permissões de acesso';
COMMENT ON TABLE usuario_grupo   IS 'Usuários vinculados a cada grupo (N:N); todo usuário tem pelo menos 1
grupo';
COMMENT ON TABLE modulo          IS 'Módulo do sistema exibido como seção (ex.: EDITAIS, CONTRATOS)';
COMMENT ON TABLE funcionalidade  IS 'Funcionalidade de um módulo (ex.: Seleção de editais)';
COMMENT ON TABLE permissao       IS 'Ação habilitável por toggle na tela de grupo (ex.: Visualizar editais)';
COMMENT ON TABLE grupo_permissao IS 'Permissões habilitadas em cada grupo (linha presente = toggle ligado)';
COMMENT ON TABLE registro_acesso IS 'Auditoria das tentativas de login';

COMMENT ON COLUMN configuracao_ad.id                IS 'Identificador fixo da linha única (sempre 1)';
COMMENT ON COLUMN configuracao_ad.tenant_id         IS 'ID do diretório (tenant) da empresa no Microsoft
Entra ID / AD';
```

```sql
COMMENT ON COLUMN configuracao_ad.client_id          IS 'ID do aplicativo da plataforma registrado no AD';
COMMENT ON COLUMN configuracao_ad.grupo_ad_object_id IS 'ID do grupo do AD cujos membros podem acessar a
plataforma';
COMMENT ON COLUMN configuracao_ad.grupo_ad_nome      IS 'Nome do grupo do AD (exibição)';
COMMENT ON COLUMN configuracao_ad.dominio_email      IS 'Domínio do e-mail corporativo';

COMMENT ON COLUMN usuario.email            IS 'E-mail corporativo; chave de login (único, sem diferenciar
maiúsculas)';
COMMENT ON COLUMN usuario.nome             IS 'Nome vindo do AD; preenchido/atualizado pela plataforma a cada
login';
COMMENT ON COLUMN usuario.ad_object_id     IS 'Identificador do usuário no AD (claim "oid" do token)';
COMMENT ON COLUMN usuario.ativo            IS 'Indica se o acesso está liberado (bloqueio sem excluir)';
COMMENT ON COLUMN usuario.ultimo_acesso_em IS 'Data/hora do último login com sucesso; NULL = nunca acessou';

COMMENT ON COLUMN papel.nome      IS 'Nome do papel exibido na coluna PAPEL';
COMMENT ON COLUMN papel.descricao IS 'Descrição do papel';
COMMENT ON COLUMN usuario_papel.papel_id IS 'Papel atribuído ao usuário';

COMMENT ON COLUMN grupo.nome      IS 'Nome do grupo ("Nome do grupo" na tela)';
COMMENT ON COLUMN grupo.descricao IS 'Descrição do grupo';
COMMENT ON COLUMN usuario_grupo.grupo_id IS 'Grupo ao qual o usuário pertence';

COMMENT ON COLUMN modulo.codigo IS 'Código estável do módulo, usado pela aplicação';
COMMENT ON COLUMN modulo.nome   IS 'Nome do módulo exibido como seção na tela de grupo';
COMMENT ON COLUMN funcionalidade.modulo_id IS 'Módulo ao qual a funcionalidade pertence';
COMMENT ON COLUMN funcionalidade.codigo    IS 'Código estável da funcionalidade, usado pela aplicação';
COMMENT ON COLUMN funcionalidade.nome      IS 'Nome exibido como título do bloco de toggles';
COMMENT ON COLUMN permissao.funcionalidade_id      IS 'Funcionalidade à qual a permissão pertence';
COMMENT ON COLUMN permissao.codigo                 IS 'Código verificado pelo back-end (ex.:
EDITAL_VISUALIZAR)';
COMMENT ON COLUMN permissao.nome                   IS 'Texto exibido ao lado do toggle';
COMMENT ON COLUMN permissao.permissao_requisito_id IS 'Permissão que precisa estar habilitada antes desta';
COMMENT ON COLUMN grupo_permissao.grupo_id     IS 'Grupo que recebe a permissão';
COMMENT ON COLUMN grupo_permissao.permissao_id IS 'Permissão habilitada no grupo';

COMMENT ON COLUMN registro_acesso.usuario_id      IS 'Usuário identificado; NULL se o e-mail não estiver
cadastrado';
COMMENT ON COLUMN registro_acesso.email_informado IS 'E-mail retornado pelo AD na tentativa';
COMMENT ON COLUMN registro_acesso.resultado       IS 'SUCESSO, NAO_CADASTRADO, FORA_GRUPO_AD ou INATIVO';
COMMENT ON COLUMN registro_acesso.endereco_ip     IS 'IP de origem da tentativa';
COMMENT ON COLUMN registro_acesso.user_agent      IS 'Navegador/dispositivo de origem';
COMMENT ON COLUMN registro_acesso.ocorrido_em     IS 'Data/hora da tentativa';

-- Comentários padronizados das colunas técnicas repetidas
DO $$
DECLARE c record;
BEGIN
  FOR c IN
    SELECT table_name, column_name FROM information_schema.columns
     WHERE table_schema = 'controle_acesso'
       AND column_name IN ('id','criado_em','atualizado_em','criado_por_id',
                           'atualizado_por_id','concedida_em','concedida_por_id',
                           'ordem','usuario_id')
       AND col_description(format('controle_acesso.%I', table_name)::regclass, ordinal_position) IS NULL
  LOOP
    EXECUTE format('COMMENT ON COLUMN %I.%I IS %L', c.table_name, c.column_name,
      CASE c.column_name
        WHEN 'id'               THEN 'Chave primária (identificador gerado automaticamente)'
        WHEN 'criado_em'        THEN 'Data/hora de criação do registro'
        WHEN 'atualizado_em'    THEN 'Data/hora da última alteração (mantida por trigger)'
        WHEN 'criado_por_id'    THEN 'Usuário que criou o registro'
        WHEN 'atualizado_por_id' THEN 'Usuário que fez a última alteração'
        WHEN 'concedida_em'     THEN 'Data/hora em que a permissão foi habilitada'
        WHEN 'concedida_por_id' THEN 'Usuário que habilitou a permissão'
        WHEN 'ordem'            THEN 'Ordem de exibição na tela'
        WHEN 'usuario_id'       THEN 'Usuário do vínculo'
      END);
```

```
  END LOOP;
END $$;
```

Apêndice B — Dados de exemplo (`02_dados_exemplo.sql`)

```sql
-- ============================================================================
--  Dados de exemplo (os mesmos nomes que aparecem nas telas do enunciado)
-- ============================================================================
SET search_path TO controle_acesso;

BEGIN;

INSERT INTO configuracao_ad (tenant_id, client_id, grupo_ad_object_id, grupo_ad_nome, dominio_email)
VALUES ('6f1d2c3b-0000-4000-8000-000000000001', '6f1d2c3b-0000-4000-8000-000000000002',
        '6f1d2c3b-0000-4000-8000-000000000003', 'GRP-PLATAFORMA-EDITAIS', 'positivo.com.br');

-- Catálogo de funcionalidades ---------------------------------------
INSERT INTO modulo (codigo, nome, ordem) VALUES
  ('EDITAIS',    'Editais',    1),
  ('CONTRATOS',  'Contratos',  2),
  ('FINANCEIRO', 'Financeiro', 3);

INSERT INTO funcionalidade (modulo_id, codigo, nome, ordem)
SELECT m.id, f.codigo, f.nome, f.ordem
  FROM (VALUES ('EDITAIS',    'SELECAO_EDITAIS',    'Seleção de editais',    1),
               ('EDITAIS',    'ANALISE_EDITAIS',    'Análise de editais',    2),
               ('CONTRATOS',  'CONSULTA_CNPJ',      'Consulta de CNPJ',      1),
               ('CONTRATOS',  'CADASTRO_CONTRATOS', 'Cadastro de contratos', 2),
               ('FINANCEIRO', 'CONTAS_RECEBER',     'Contas a receber',      1)
       ) AS f(modulo, codigo, nome, ordem)
  JOIN modulo m ON m.codigo = f.modulo
 ORDER BY m.ordem, f.ordem;

INSERT INTO permissao (funcionalidade_id, codigo, nome, ordem)
SELECT fu.id, p.codigo, p.nome, p.ordem
  FROM (VALUES
    ('SELECAO_EDITAIS',    'EDITAL_VISUALIZAR',            'Visualizar editais',                     1),
    ('SELECAO_EDITAIS',    'EDITAL_GERENCIAR_ANEXOS',      'Gerenciar anexos do edital',             2),
    ('SELECAO_EDITAIS',    'EDITAL_GERENCIAR_STATUS',      'Gerenciar status',                       3),
    ('SELECAO_EDITAIS',    'EDITAL_ADICIONAR_MANUAL',      'Adicionar edital manualmente',           4),
    ('SELECAO_EDITAIS',    'EDITAL_GERENCIAR_GRUPOS',      'Gerenciar grupos de editais',            5),
    ('ANALISE_EDITAIS',    'ANALISE_GERENCIAR',            'Gerenciar análise de edital',            1),
    ('ANALISE_EDITAIS',    'ANALISE_VISUALIZAR_HISTORICO', 'Visualizar histórico de análise de edital',2),
    ('ANALISE_EDITAIS',    'ANALISE_PRE_TECNICA',          'Realizar pré-análise técnica',           3),
    ('ANALISE_EDITAIS',    'ANALISE_REALIZAR',             'Realizar análise de edital',             4),
    ('ANALISE_EDITAIS',    'ANALISE_GERENCIAR_ETAPAS',     'Gerenciar etapas do edital',             5),
    ('ANALISE_EDITAIS',    'ANALISE_REALIZAR_PROPOSTA',    'Realizar proposta',                      6),
    ('ANALISE_EDITAIS',    'ANALISE_GERENCIAR_HISTORICO',  'Gerenciar histórico de análise de edital',7),
    ('CONSULTA_CNPJ',      'CNPJ_GERENCIAR_CONSULTA',      'Gerenciar consulta',                     1),
    ('CADASTRO_CONTRATOS', 'CONTRATO_VISUALIZAR',          'Visualizar contratos',                   1),
    ('CADASTRO_CONTRATOS', 'CONTRATO_CADASTRAR',           'Cadastrar contrato',                     2),
    ('CADASTRO_CONTRATOS', 'CONTRATO_EDITAR',              'Editar contrato',                        3),
    ('CADASTRO_CONTRATOS', 'CONTRATO_EXCLUIR',             'Excluir contrato',                       4),
    ('CONTAS_RECEBER',     'RECEBER_VISUALIZAR',           'Visualizar contas a receber',            1),
    ('CONTAS_RECEBER',     'RECEBER_REGISTRAR',            'Registrar recebimento',                  2)
  ) AS p(funcionalidade, codigo, nome, ordem)
  JOIN funcionalidade fu ON fu.codigo = p.funcionalidade
 ORDER BY fu.id, p.ordem;

-- Dependências entre permissões (toggles desabilitados na tela)
UPDATE permissao p
   SET permissao_requisito_id = r.id
  FROM (VALUES ('EDITAL_GERENCIAR_ANEXOS',     'EDITAL_VISUALIZAR'),
               ('EDITAL_GERENCIAR_STATUS',     'EDITAL_VISUALIZAR'),
               ('EDITAL_ADICIONAR_MANUAL',     'EDITAL_VISUALIZAR'),
               ('EDITAL_GERENCIAR_GRUPOS',     'EDITAL_VISUALIZAR'),
               ('ANALISE_GERENCIAR_HISTORICO', 'ANALISE_VISUALIZAR_HISTORICO'),
               ('CONTRATO_CADASTRAR',          'CONTRATO_VISUALIZAR'),
```

```
              ('CONTRATO_EDITAR',             'CONTRATO_VISUALIZAR'),
              ('CONTRATO_EXCLUIR',            'CONTRATO_VISUALIZAR'),
              ('RECEBER_REGISTRAR',           'RECEBER_VISUALIZAR')
       ) AS d(permissao, requisito)
  JOIN permissao r ON r.codigo = d.requisito
 WHERE p.codigo = d.permissao;

-- Papéis ----------------------------------------------------------------
INSERT INTO papel (nome) VALUES
  ('Analista de proposta'), ('Analista técnico'), ('Analista de documentos');

-- Grupos e suas permissões ----------------------------------------------
INSERT INTO grupo (nome) VALUES
  ('Cadastro de contratos'), ('Contas a receber'), ('Liderança de editais'),
  ('Seleção de editais'), ('Validação de CNPJ');

INSERT INTO grupo_permissao (grupo_id, permissao_id)
SELECT g.id, p.id
  FROM (VALUES
    ('Cadastro de contratos', 'CONTRATO_VISUALIZAR'),
    ('Cadastro de contratos', 'CONTRATO_CADASTRAR'),
    ('Cadastro de contratos', 'CONTRATO_EDITAR'),
    ('Contas a receber',      'RECEBER_VISUALIZAR'),
    ('Contas a receber',      'RECEBER_REGISTRAR'),
    ('Validação de CNPJ',     'CNPJ_GERENCIAR_CONSULTA'),
    ('Validação de CNPJ',     'CONTRATO_VISUALIZAR')
  ) AS x(grupo, permissao)
  JOIN grupo g     ON g.nome = x.grupo
  JOIN permissao p ON p.codigo = x.permissao;

-- Seleção de editais = todas as permissões da funcionalidade
INSERT INTO grupo_permissao (grupo_id, permissao_id)
SELECT g.id, p.id
  FROM grupo g, permissao p JOIN funcionalidade f ON f.id = p.funcionalidade_id
 WHERE g.nome = 'Seleção de editais' AND f.codigo = 'SELECAO_EDITAIS';

-- Liderança de editais = todas as permissões do módulo EDITAIS
INSERT INTO grupo_permissao (grupo_id, permissao_id)
SELECT g.id, p.id
  FROM grupo g, permissao p
  JOIN funcionalidade f ON f.id = p.funcionalidade_id
  JOIN modulo m         ON m.id = f.modulo_id
 WHERE g.nome = 'Liderança de editais' AND m.codigo = 'EDITAIS';

-- Usuários (o nome vem do AD; Guilherme ainda não acessou → nome NULL) --
INSERT INTO usuario (email, nome, ad_object_id, ultimo_acesso_em) VALUES
  ('adailton@positivo.com.br',        'Adailton Araújo',  'a0000000-0000-4000-8000-000000000001', now() -
interval '3 hours'),
  ('laishc@positivo.com.br',          'Laís Cândido',     'a0000000-0000-4000-8000-000000000002', now() -
interval '6 hours'),
  ('usuario32024@positivo.com',       'Rodrigo Mendes',   'a0000000-0000-4000-8000-000000000003', now() -
interval '30 minutes'),
  ('djuliads@positivo.com.br',        'Djulia dos Santos','a0000000-0000-4000-8000-000000000004', now() -
interval '1 day'),
  ('guilhermesousa@positivo.com.br',  NULL,               NULL,                                   NULL);

INSERT INTO usuario_grupo (usuario_id, grupo_id)
SELECT u.id, g.id
  FROM (VALUES
    ('adailton@positivo.com.br',       'Cadastro de contratos'),
    ('adailton@positivo.com.br',       'Validação de CNPJ'),
    ('adailton@positivo.com.br',       'Contas a receber'),
    ('adailton@positivo.com.br',       'Liderança de editais'),
    ('adailton@positivo.com.br',       'Seleção de editais'),
    ('laishc@positivo.com.br',         'Validação de CNPJ'),
    ('laishc@positivo.com.br',         'Seleção de editais'),
    ('laishc@positivo.com.br',         'Contas a receber'),
    ('usuario32024@positivo.com',      'Cadastro de contratos'),
```

```
    ('usuario32024@positivo.com',      'Validação de CNPJ'),
    ('djuliads@positivo.com.br',       'Cadastro de contratos'),
    ('djuliads@positivo.com.br',       'Seleção de editais'),
    ('guilhermesousa@positivo.com.br', 'Cadastro de contratos'),
    ('guilhermesousa@positivo.com.br', 'Seleção de editais')
  ) AS x(email, grupo)
  JOIN usuario u ON u.email = x.email
  JOIN grupo g   ON g.nome = x.grupo;

INSERT INTO usuario_papel (usuario_id, papel_id)
SELECT u.id, p.id
  FROM (VALUES
    ('adailton@positivo.com.br', 'Analista de proposta'),
    ('adailton@positivo.com.br', 'Analista técnico'),
    ('adailton@positivo.com.br', 'Analista de documentos'),
    ('laishc@positivo.com.br',   'Analista de proposta'),
    ('usuario32024@positivo.com','Analista de documentos'),
    ('djuliads@positivo.com.br', 'Analista técnico')
  ) AS x(email, papel)
  JOIN usuario u ON u.email = x.email
  JOIN papel p   ON p.nome = x.papel;

INSERT INTO registro_acesso (usuario_id, email_informado, resultado, endereco_ip, ocorrido_em)
SELECT id, email, 'SUCESSO', '10.0.0.10', ultimo_acesso_em
  FROM usuario WHERE ultimo_acesso_em IS NOT NULL;

INSERT INTO registro_acesso (usuario_id, email_informado, resultado, endereco_ip, ocorrido_em)
VALUES (NULL, 'visitante@gmail.com', 'NAO_CADASTRADO', '200.1.2.3', now() - interval '2 hours');

COMMIT;
```